IE540
L14. AlphaGo, Reward Shaping

KAIST ISE
Hayong Shin

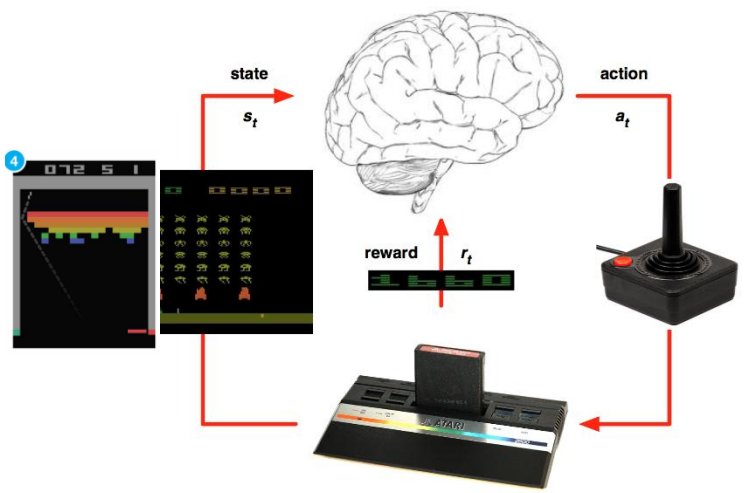
Google Deepmind

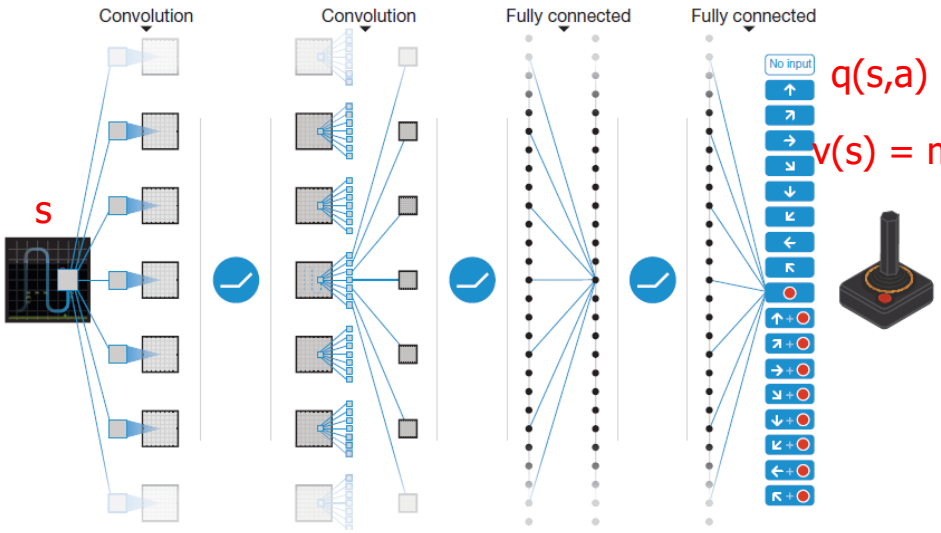
- In search of AGI : Artificial General Intelligence, like human
  - Instead of Narrow AI (hand-crafted software only for specific task)
    - Deep Blue, which defeated human champion of chess
- 2015 Nature paper : Playing 49 Atari video games like human
  - DRL (Deep Reinforcement Learning)
    - **Reinforcement learning** framework

    + **Deep learning** for approximating value function

  - Without any prior knowledge about the games

  → Huge success in almost all games

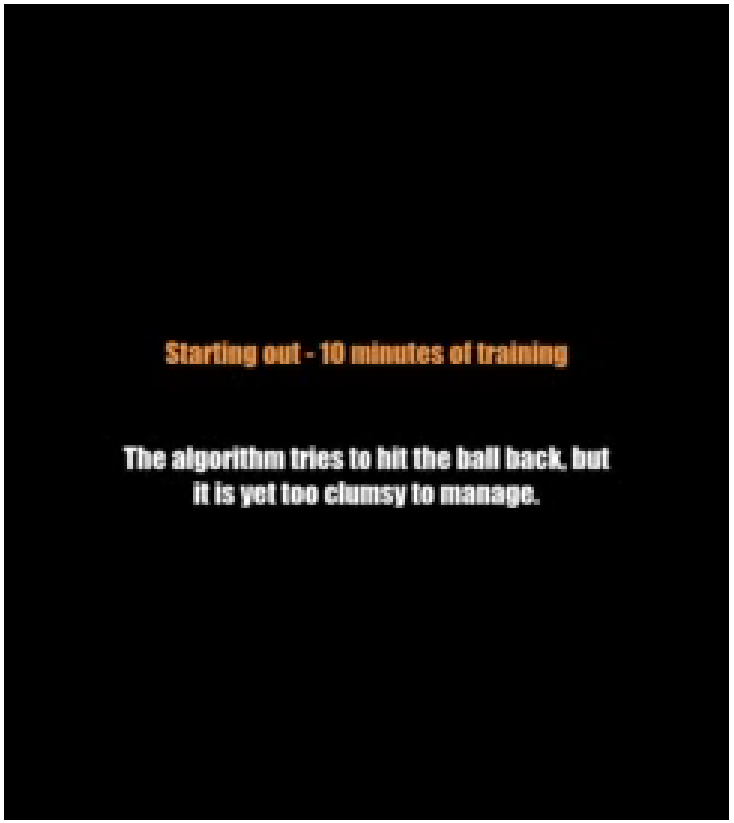




Details of DQN Atari game

- Input : sequence of screen images
  - Original image : 210x160, 128 colors
    → 84x84 grayscale image
  - Stack 4 most recent images

  → 84x84x4 images
- Network architecture : 3 conv + 2 FC
  - Conv. & downsample → 20x20x32
  - Conv. & downsample → 9x9x64
  - Conv. → 7x7x64 → flatten to 3136
  - FC → 512 hidden units
  - FC → 18 outputs (action values)
- Experience replay
  - Mini-batch w/ RMSProp
- Reward : game score change (-1, 0, +1)
- ε-greedy (decreasing as 1/n)
- Fixed Q (fixed weight)
- TD error clipping (-1~+1)

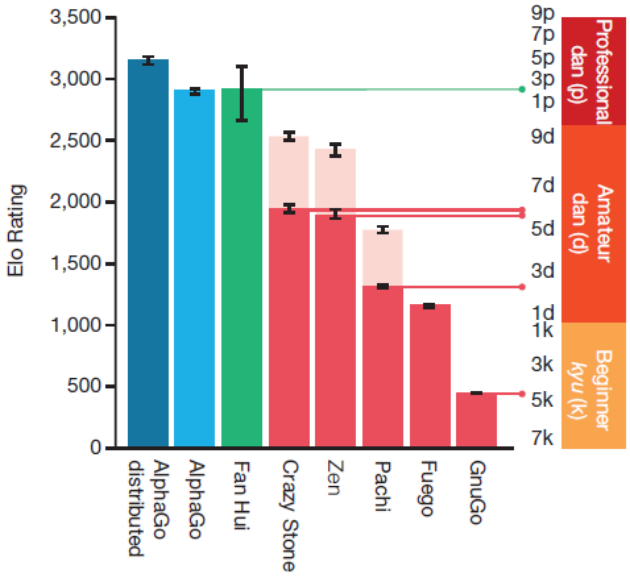


What is "Go" game?



- Baduk(Korea), Go (Japan), Weiqi (China)
- "Most" complex board game
  - 19x19 board : much bigger state space than that of chess
    - Width (possible moves) ≈ 250 (cf. 35 in chess)
    - Depth (# of steps) ≈ 150 (cf. 80 in chess)
  - Player with larger territory wins
- 5000+ years of history

Rank Type	Range	Stage
Double-digit *kyu*	30–21k	Beginner
Double-digit *kyu*	20–10k	Casual player
Single-digit *kyu*	9–1k	Intermediate/club player
Amateur *dan*	1–7d (where 8d is a special title)	Advanced player
Professional *dan*	1–9p (where 10p is a special title)	Professionals



AlphaGo by Google Deepmind



- **Deep Reinforcement Learning**
  - **Deep learning** (Deep Neural Net; DNN)
    - **next move selection (policy function)**
    - **state evaluation (value function)**
  - **Reinforcement learning**
    - Used for training DNN through self-play
- No expert (professional) knowledge of 'Go game'
  - Self-play → **Simulation**



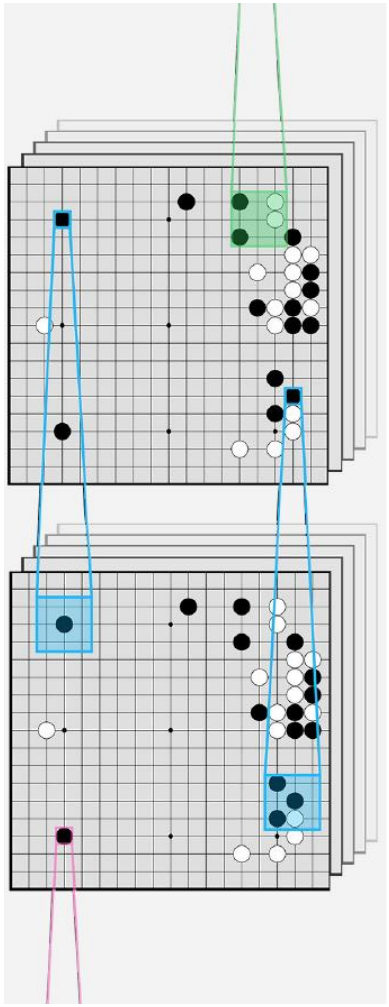


AlphaGo – Components

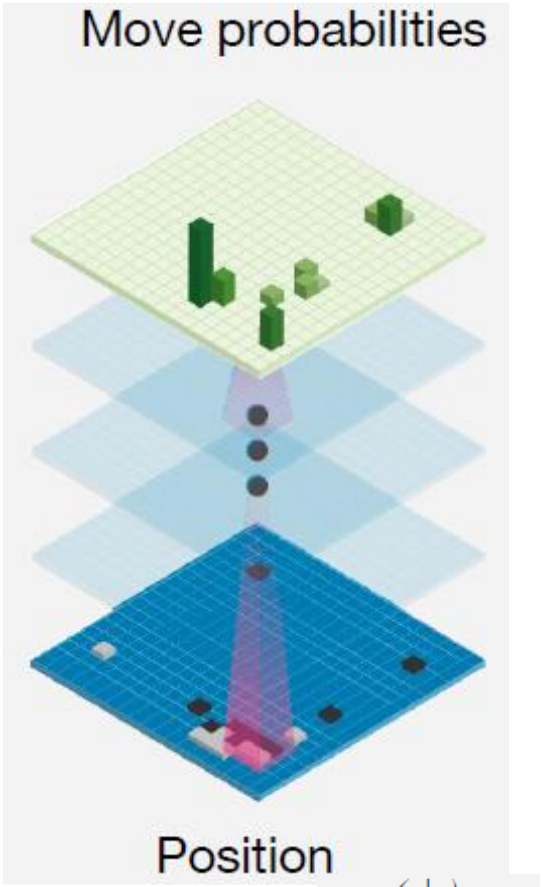
- **Deep learning**
  - **Policy Net : DNN for Next move recommendation (착수선택)**
    - Recommend moves most frequently played
    - Used for reducing breadth of search
  - **Value Net : DNN for State Evaluation (형세판단)** $v_\theta(.)$
    - Predict probability of winning at current state
    - Used for reducing depth of search
  - No hand crafted feature (such as common sequence (정석))
    - **Raw input is used** : board configuration = 19x19 image
- **Reinforcement learning**
  - Self-play → simulation : numerous quality data
  - Used ultimate reward (**Win or Lose**)
    - not intermediate measure (# of captured stone, area of secured territory, …)
- **MCTS (Monte Carlo Tree Search) : search for next move (수읽기)**
  - Huge search space (prohibitively large, 10^{40})
    - Breadth reduction by Policy Net
    - Depth reduction by Value Net : Combine value net + rollout
      - **Rollout** : Play to the end for some provable alternatives while time permits

Policy net & Value net

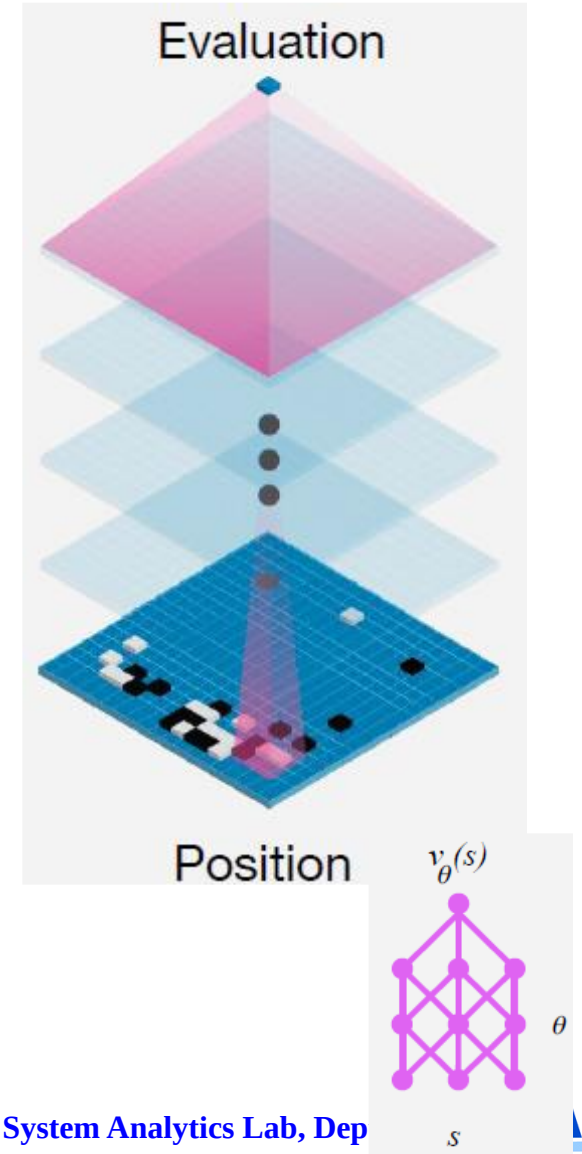
- Convolutional Neural Net



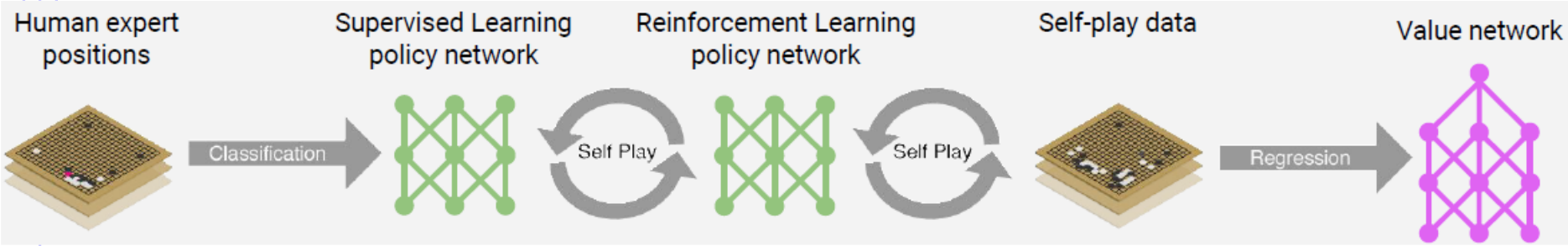
- Policy net



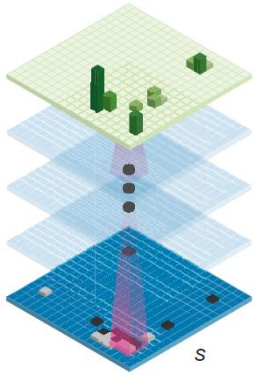
- Value net

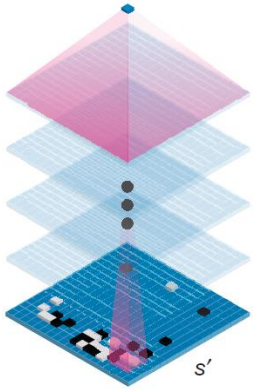


Training AlphaGo

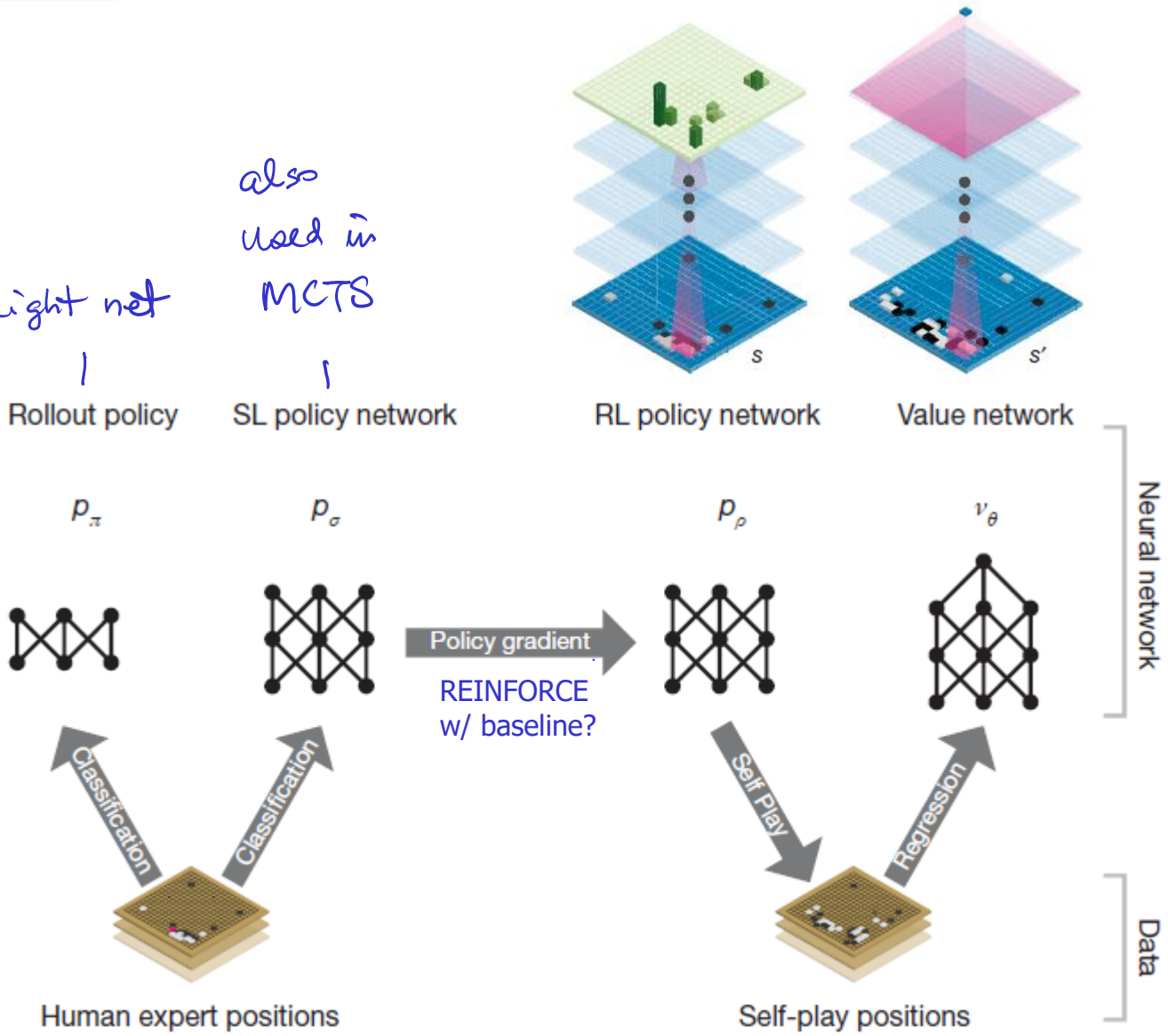


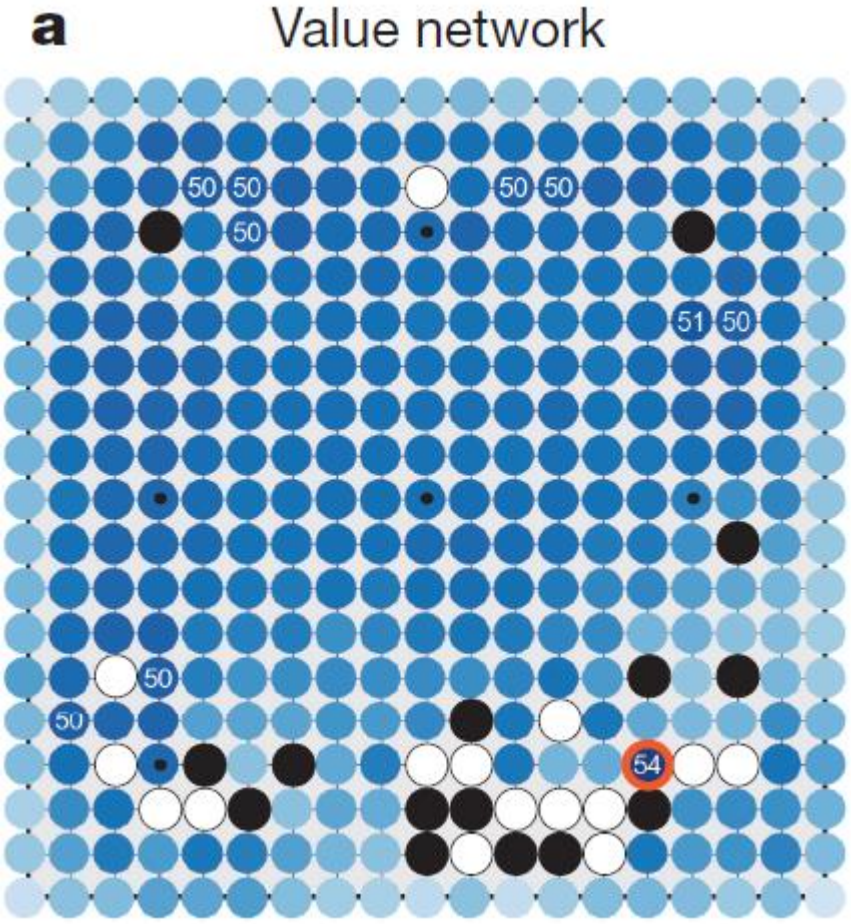
- Input : 19x19x48 image stack, Net arch.: 13 conv. layers + 1 FC layer
- Policy Net (initial supervised learning) : P_σ - policy from SL
  - Training : 30M moves records (from 160K games)
    - Amateur level game records from Internet Go Site (KGS data)
    - 3 weeks using 50 CPUs
  - 57% accuracy of prediction : weak amateur level (7k?)
- Policy Net (RL, Policy gradient) : P_ρ - policy from RL
  - Initial Policy net : P_σ, PG through Self-play (w/o MCTS)
  - Strong amateur level (amateur 3d?)
- Value Net : state evaluation $v_\theta(.)$
  - High quality data set (30M positions) from self-plays w/ P_σ
  - 50 CPUs, 1week → $v_\theta(.)$, stronger than P_σ (amateur 4~5d?)
- MCTS : $v_\theta(.)$+ Rollout (rollout policy P_π) → World best

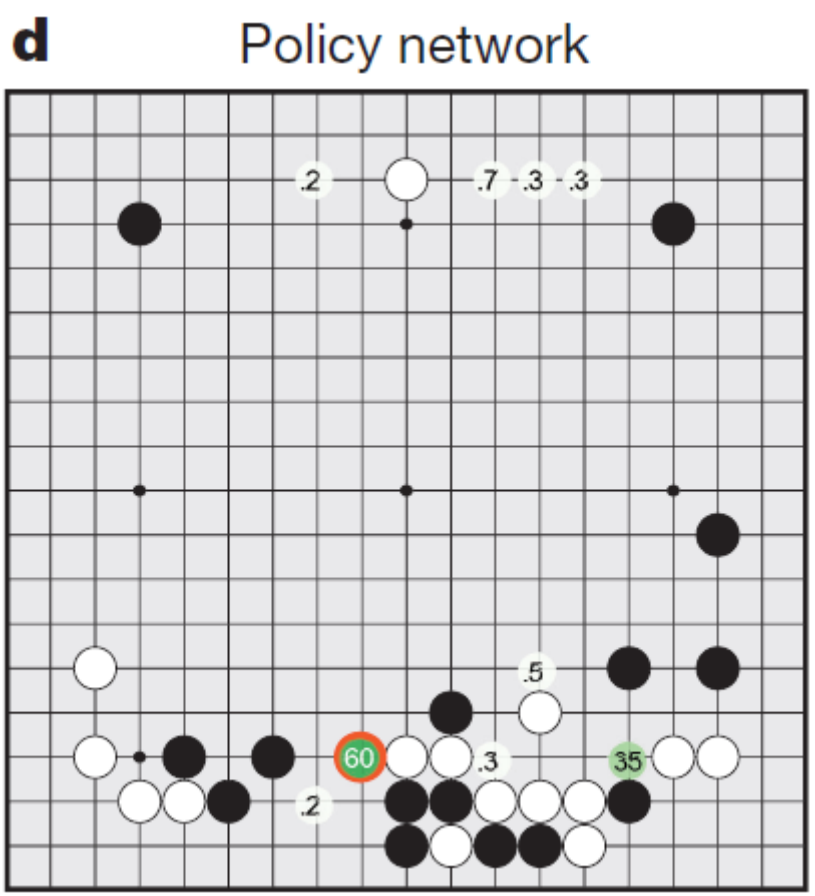




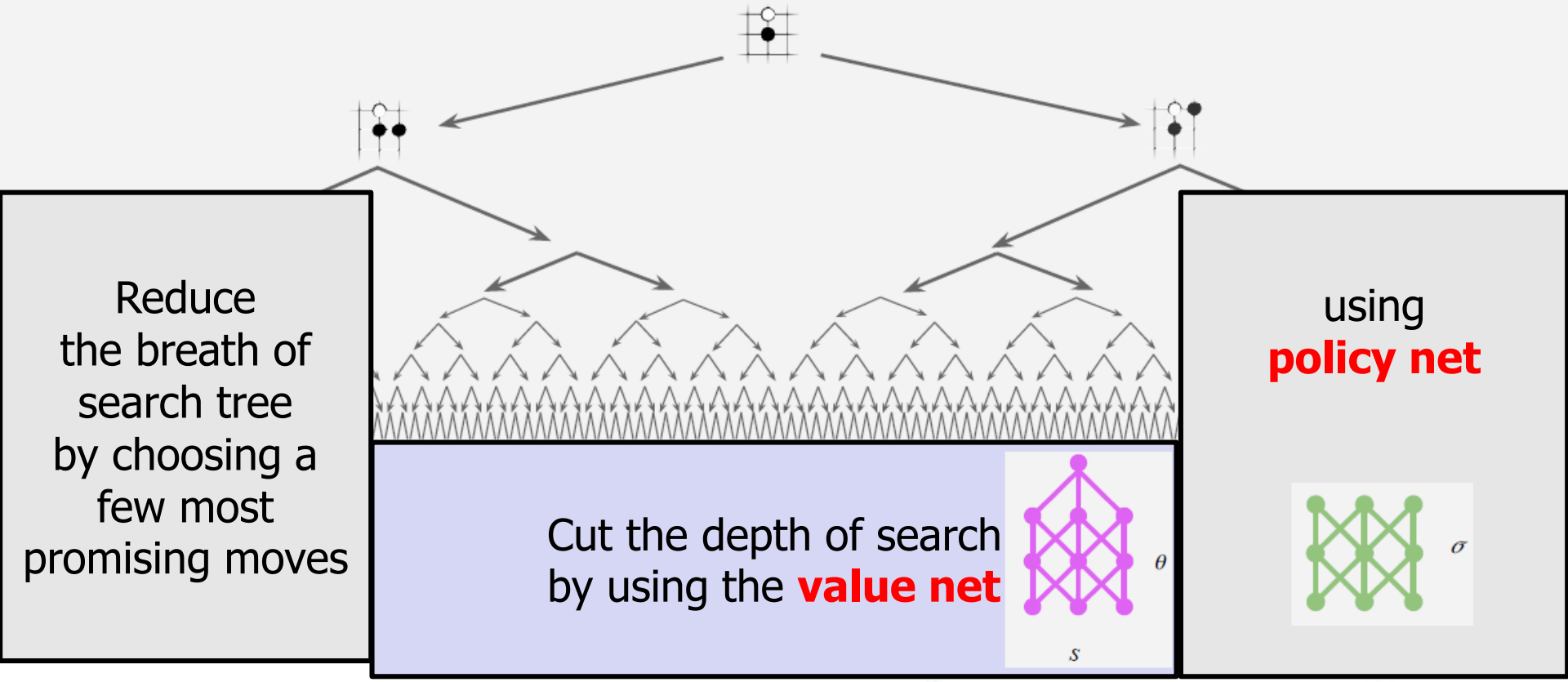
Training AlphaGo




a
Value network
50
50
50
50
50
51
50
50
50
54


d
Policy network
.2
.7
.3
.3
.5
60
.3
35
.2

How does AlphaGo select the next move? : Tree Search



Reduce the breath of search tree by choosing a few most promising moves

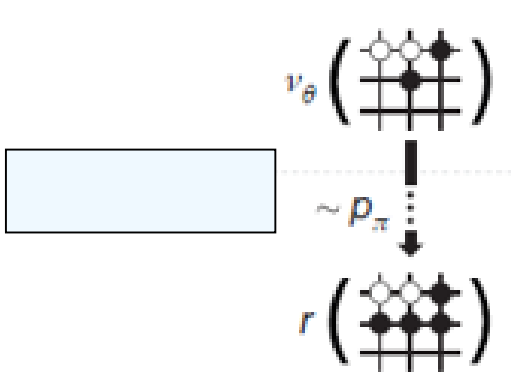
Cut the depth of search by using the **value net**

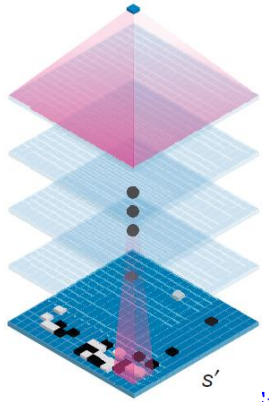
θ

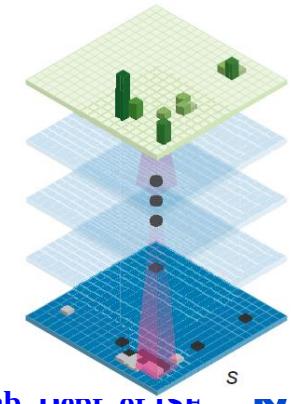
s

using **policy net**

σ



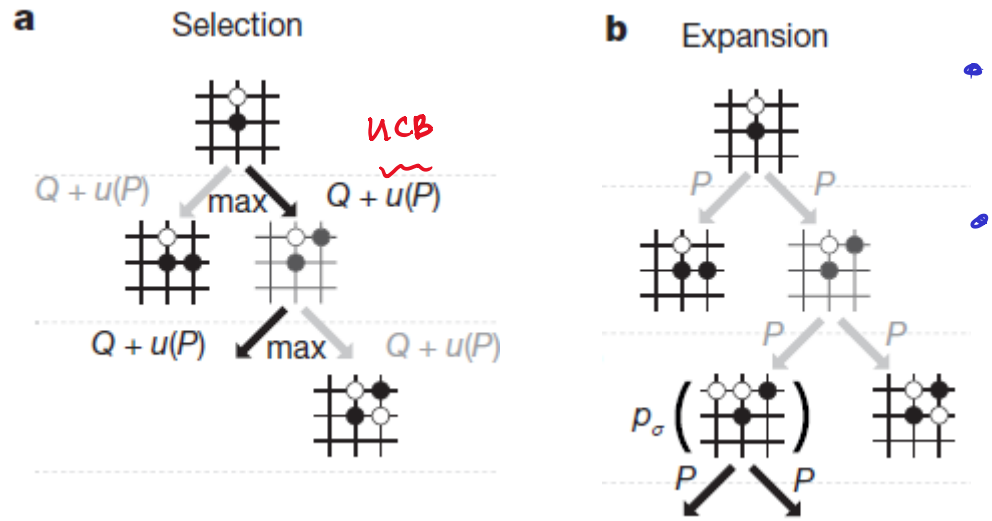


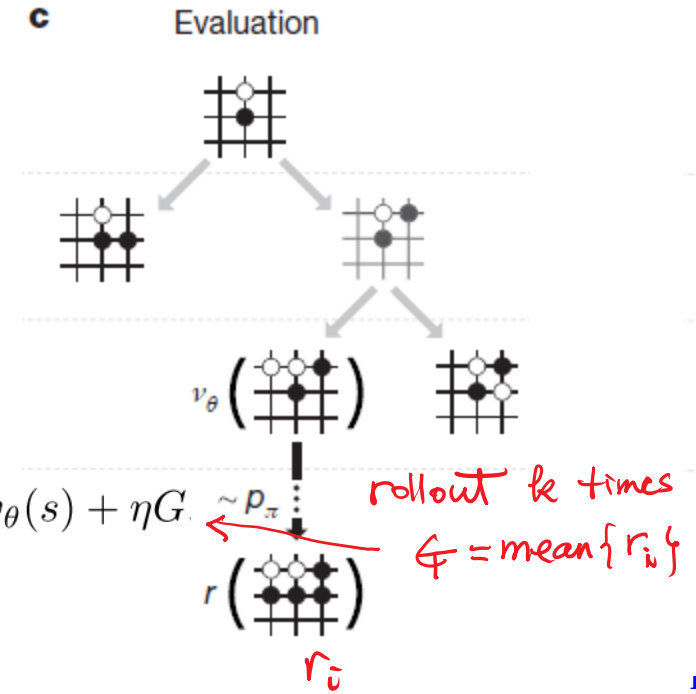


MCTS + Rollout

- P_σ: policy from SL (of KGS data)
  → used for < initializing P_θ / MCTS
- P_θ: policy from RL
  → used for generating training data for VG
- P_π: light-not policy from SL
  → used for rollout in MCTS

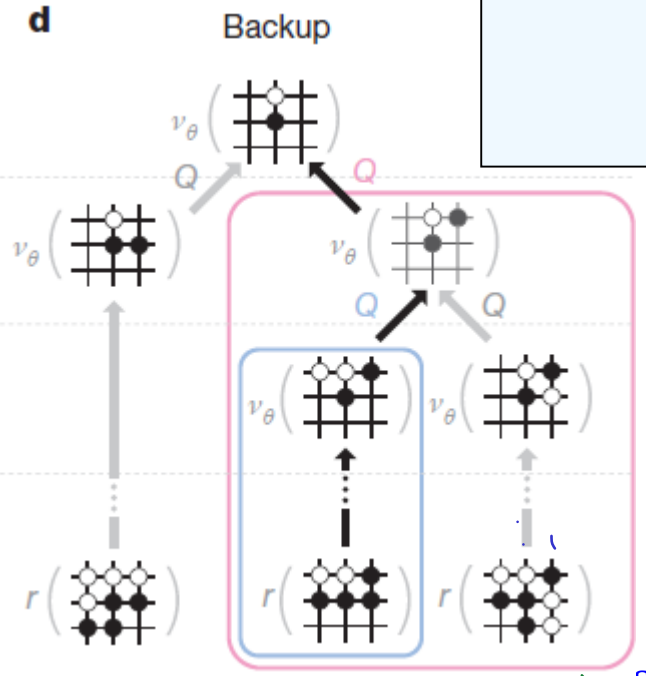
TAQ. Why P_σ is used in MCTS (not P_θ)?



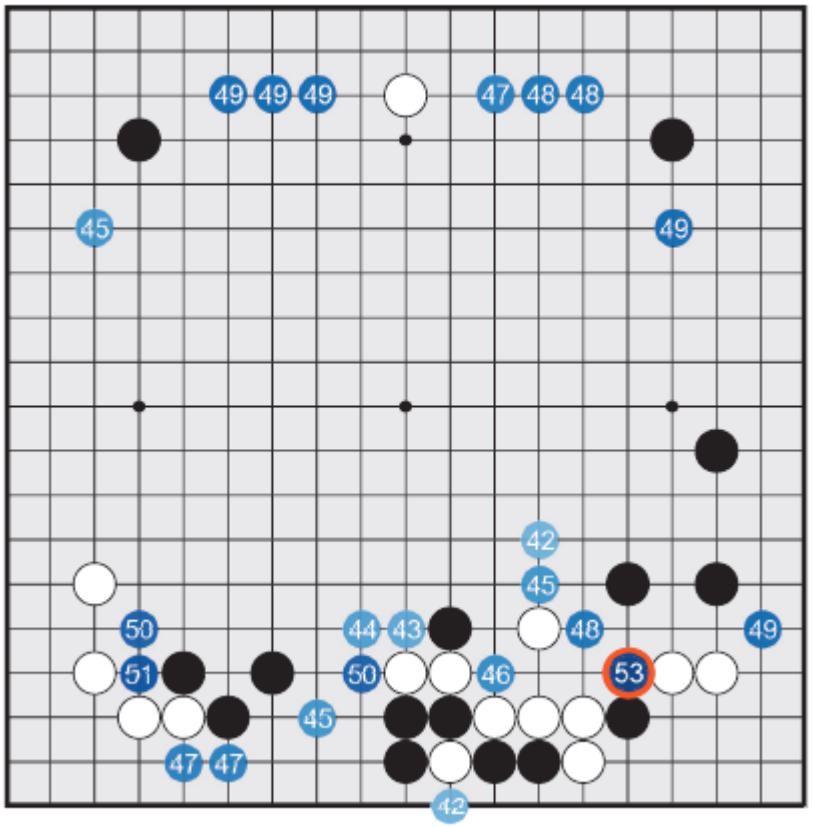


$$v(s) = (1-\eta)v_\theta(s) + \eta G$$

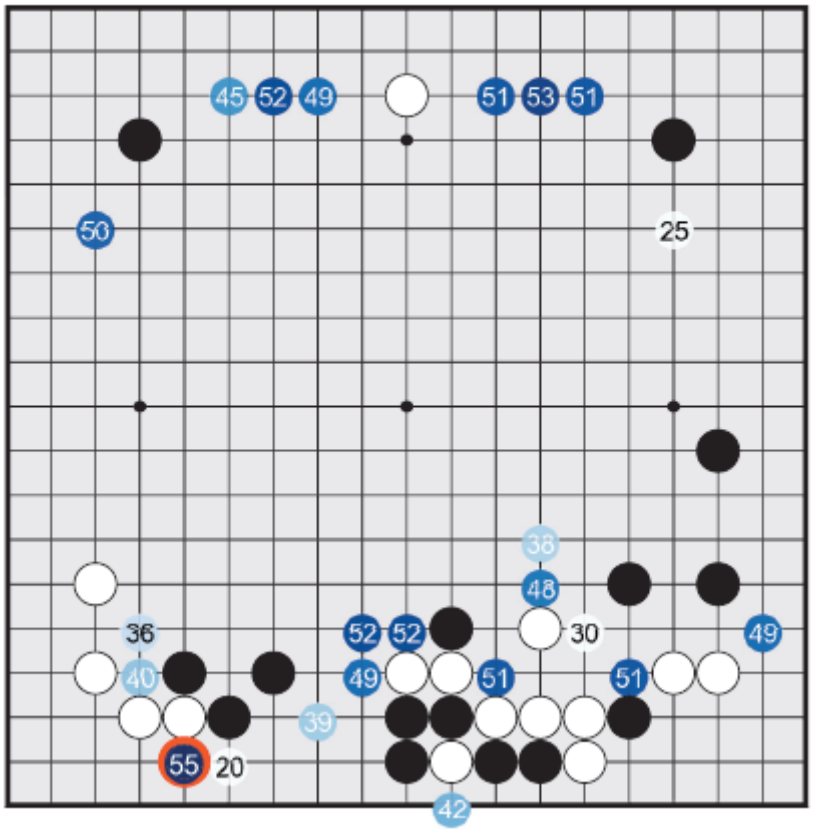
$\eta = 0.5$



b Tree evaluation from value net



c Tree evaluation from rollouts



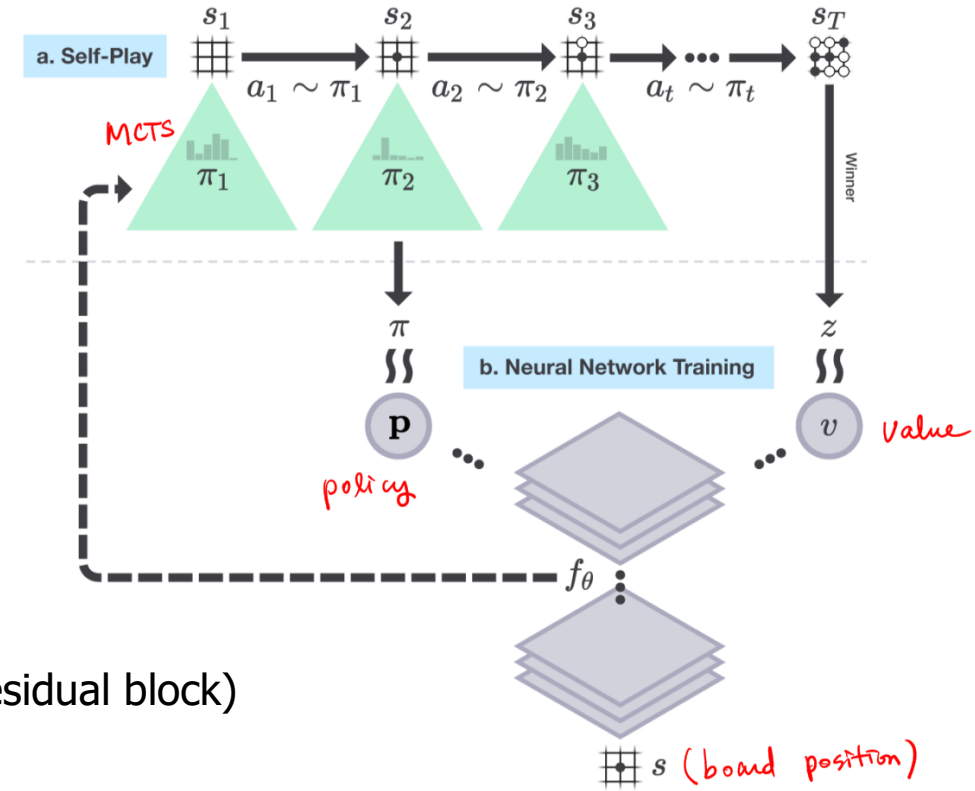
TAQ. If we had a perfect value net $v_\theta(.)$, how would its prediction output look?

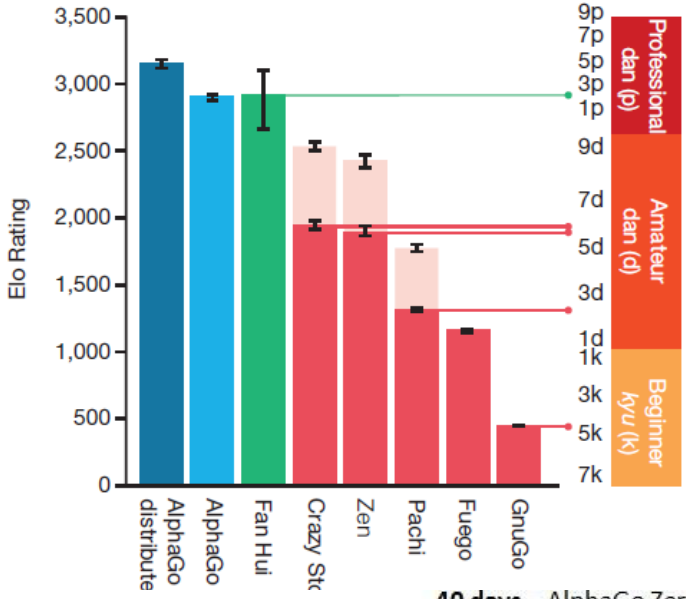
AlphaGo Zero (2017, Nature)

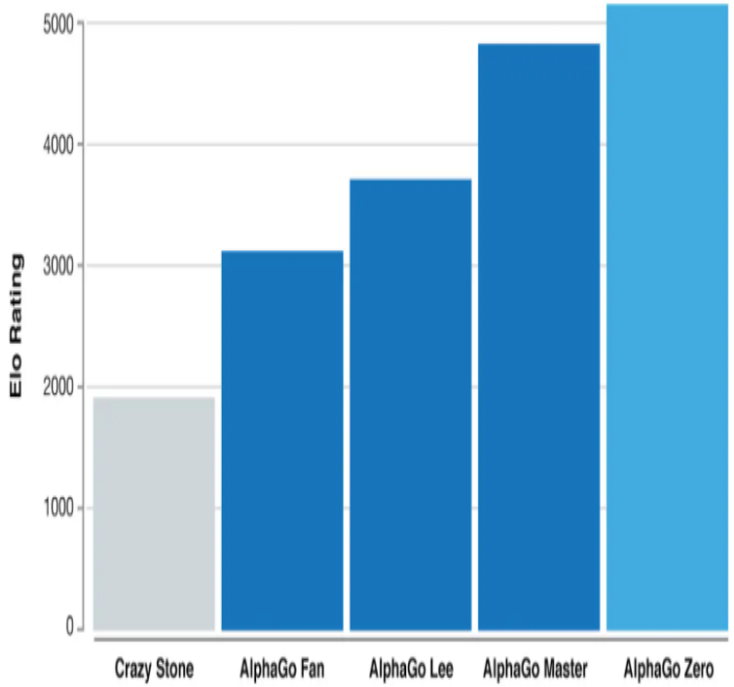
Mastering the game of Go without human knowledge

David Silver[1]*, Julian Schrittwieser[1]*, Karen Simonyan[1]*, Ioannis Antonoglou[1], Aja Huang[1], Arthur Guez[1], Thomas Hubert[1], Lucas Baker[1], Matthew Lai[1], Adrian Bolton[1], Yutian Chen[1], Timothy Lillicrap[1], Fan Hui[1], Laurent Sifre[1], George van den Driessche[1], Thore Graepel[1] & Demis Hassabis[1]

- No human play data
- Trained purely on self-play
- Single DNN, dual heads
  - Policy vector $P(s)$
  - Value $v_\theta(s)$
- MCTS used for training as well
  - Policy improvement
- No rollout
- Input : 19x19x17 (cf 48 in 2016)
  - (Current + Past 7)*2 players+1
- Net :
  - features: 41 conv.+skip conn. (residual block)
  - policy part : 43 layers
  - value part : 44 layers
- Trained w/ 4.9M self-play in 3 days
  - 100:0 vs AlphaGo-Lee
- Trained w/ 20M self-play in 40 days (bigger net)
  - 89:11 vs AlphaGo-Master, which uses human data

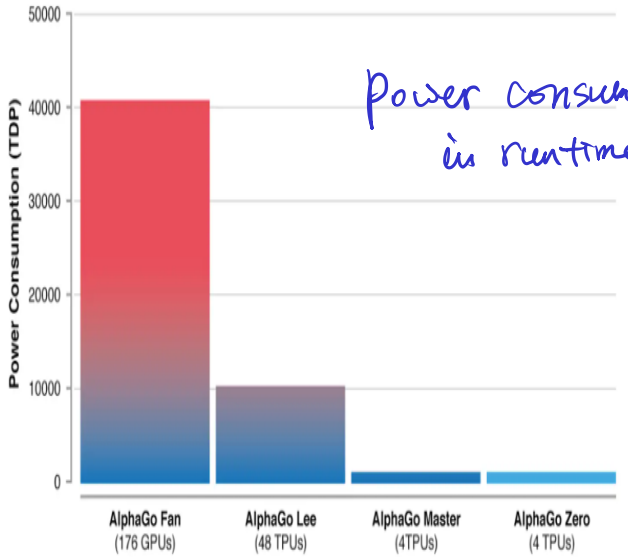






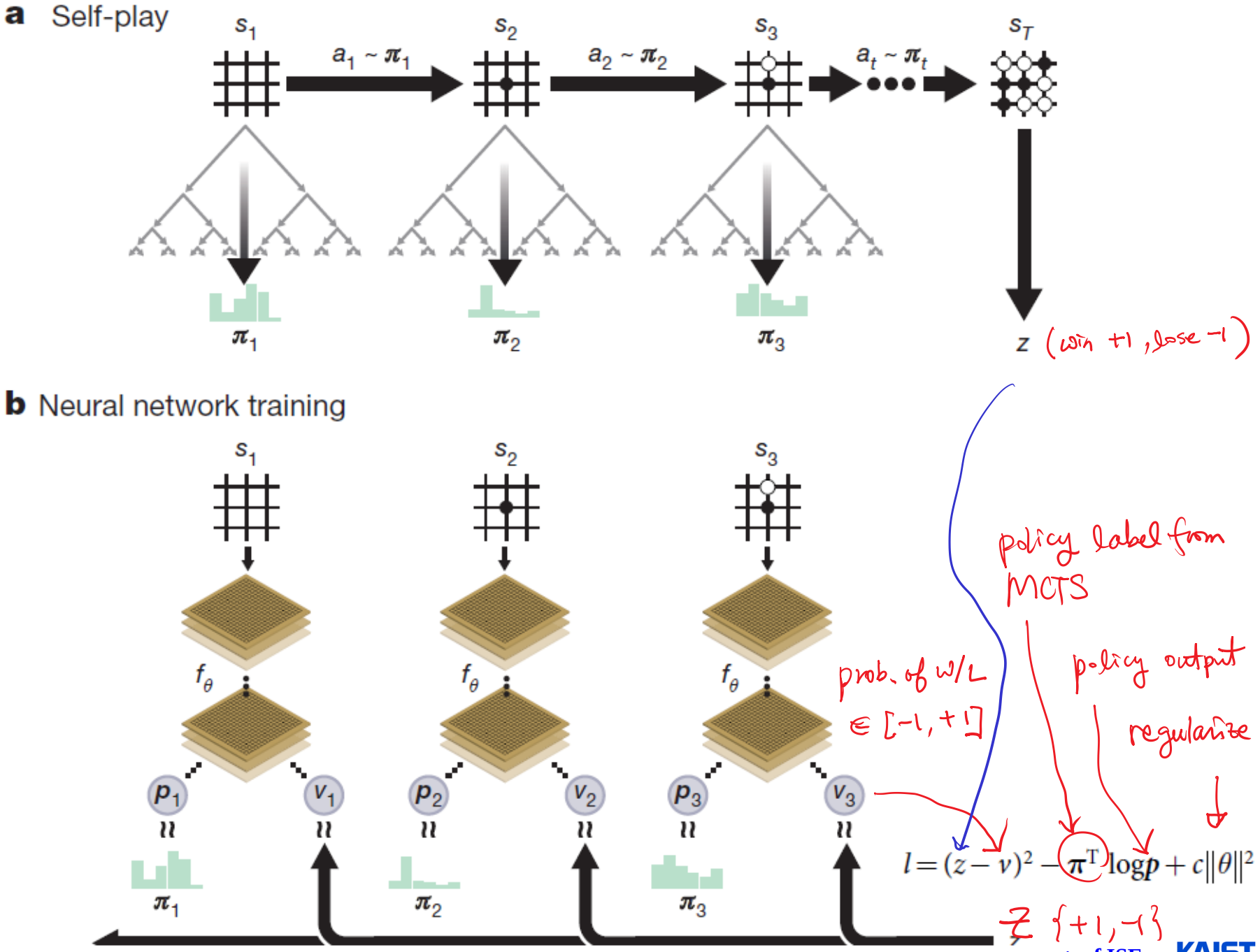
FanHui Lee Master Zero





Power consumption in runtime

FanHui Lee Master Zero


a Self-play
s_1
$a_1 \sim \pi_1$
s_2
$a_2 \sim \pi_2$
s_3
$a_t \sim \pi_t$
s_T
π_1
π_2
π_3
z (win +1, lose −1)
b Neural network training
s_1
s_2
s_3
f_θ
p_1
v_1
p_2
v_2
p_3
v_3
π_1
π_2
π_3
prob. of W/L
$\in [-1, +1]$
policy label from MCTS
policy output
regularize
$l = (z - v)^2 - \pi^T \log p + c\|\theta\|^2$
$z \in \{+1, -1\}$

Then, what?

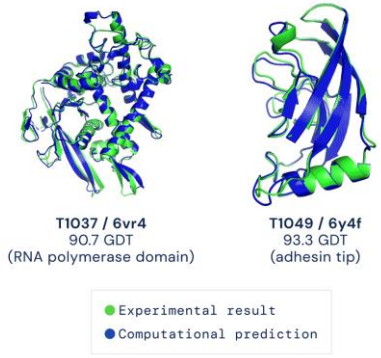
- Many computer baduk(go) programs followed the path of AlphaGo Zero
  - Leela Zero, ELF Go, …
  - Katago : added more go-specific features
    - predicting territory
- Deepmind generalized the algorithm of AlphaGo Zero
  - AlphaZero (Science, 2018)
    - generalize AlphaGo Zero to other games
    - Go, Chess, Shogi
  - MuZero (Nature, 2020)
    - more general arch. for planning (searching)
    - Atari, Go, Chess, shogi
  - AlphaFold (Nature, 2020): different from AlphaZero
    - predicting 3D structure of protein
    - from its amino-acid sequence
    - won CASP challenge in 2018, 2020

A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play

Silver *et al.*, *Science* **362**, 1140–1144 (2018) 7 December 2018

Mastering Atari, Go, chess and shogi by planning with a learned model

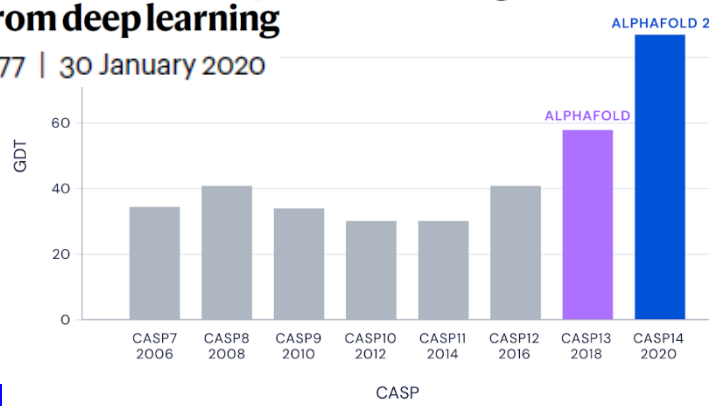
Nature | Vol 588 | 24/31 December 2020



Median Free-Modelling Accuracy

Improved protein structure prediction using potentials from deep learning

Nature | Vol 577 | 30 January 2020



Hayong Sl

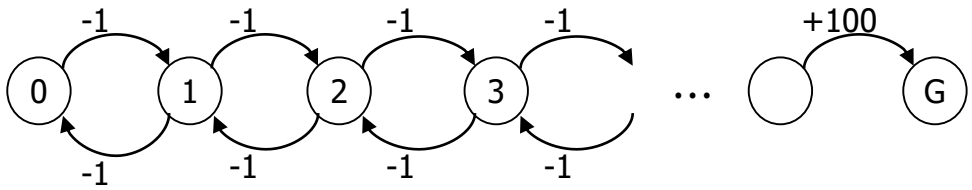
Reward shaping [1,2]

- Reward shaping
  - changing reward function
    - hoping to learn a good (or optimal) policy faster
  - Reward shaping $M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ ➔ $M' = \langle \mathcal{S}, \mathcal{A}, P, R', \gamma \rangle$
    - $R(s, a, s')$: reward function from $(s, a) \rightarrow s'$
    - $R'(s, a, s') = R(s, a, s') + F(s, a, s')$
      - $F(s, a, s')$: shaping reward function
    - ex1: to encourage moving toward goal
      - $F(s, a, s') = \mathbf{1}(s' \text{ is closer to goal than } s \text{ is})$
    - ex2: to encourage taking action a_1 when $s \in \mathcal{S}_0$
      - $F(s, a, s') = \mathbf{1}(s \in \mathcal{S}_0 \text{ and } a = a_1)$
    - ex3: making more aggressive move in cartpole (angle reward)
      - $F(s, a, s') = 10 * |s'[2]|$, where $s'[2]$ is pole angle
  - Reward shaping may alter (mislead) agent behavior in general
    - π^*_M (optimal policy for M) ≠ $\pi^*_{M'}$ (optimal policy for M')
    - policy-invariant reward shaping? ($\pi^*_{M'}$ is also optimal in M)

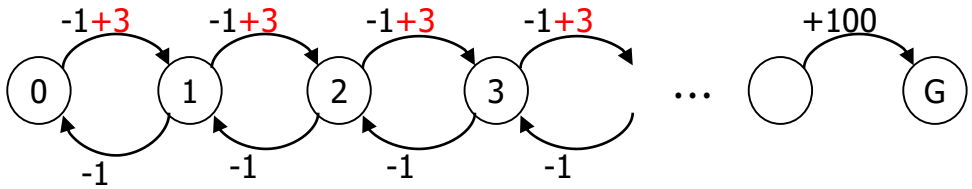
[1] A. Ng et al, "Policy invariance under reward transformations : Theory and applications to reward shaping", ICML 1999
[2] A. Ng, *Shaping and Policy Search in Reinforcement Learning*, PhD Dissertation, UC Berkeley, 2003 (adv. by M. Jordan)

example : bicycle riding

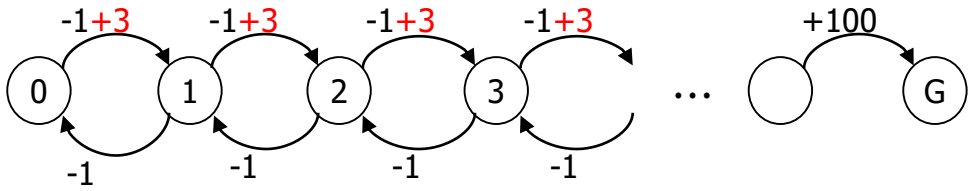
- riding bike to goal : delayed reward ➔ may not reach goal
  - state = 0, 1, 2, …, G : current position of bicycle, G can be very large
  - reward = -1 (effort) every step, +100 (delight) at goal state



- shaping reward 1 : +3 for right (closer to goal)
  - $F(s, a, s') = 3 \cdot \mathbf{1}(s' > s)$ ➔ short circling, not optimal



- shaping reward 1 : +3 for right
  - $F(s, a, s') = 3(s' - s)$



Potential based reward shaping

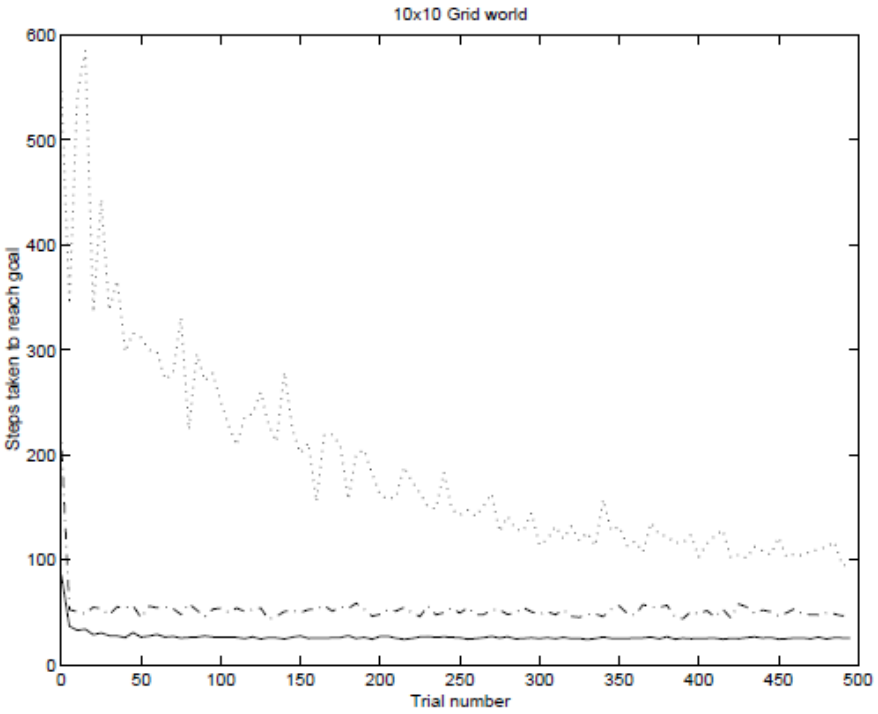
- potential based shaping reward $M' = \langle \mathcal{S}, \mathcal{A}, P, R + F, \gamma \rangle$
  - $\Phi(s)$: (real valued) potential function
  - $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$: difference in potential
- shifted value function
  - $Q^{\pi}_{M'}(s,a) = Q^{\pi}_{M}(s,a) - \Phi(s)$, $V^{\pi}_{M'}(s) = V^{\pi}_{M}(s) - \Phi(s)$

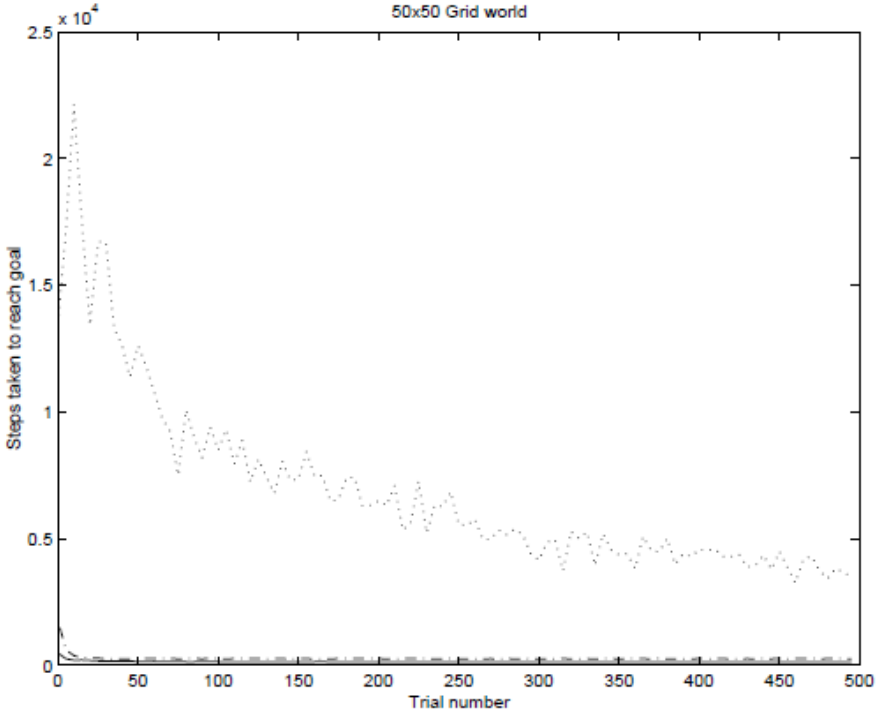
$$Q^{\pi}_{M'}(s_t, a_t) = \mathbb{E}_{\pi}[r'_t + \gamma r'_{t+1} + \gamma^2 r'_{t+2} + \cdots | s_t, a_t],\ r'_t = r_t + \gamma\Phi(s_{t+1}) - \Phi(s_t)$$
$$= \mathbb{E}_{\pi}\left[r_t + \gamma\Phi(s_{t+1}) - \Phi(s_t) + \gamma\left(r_{t+1} + \gamma\Phi(s_{t+2}) - \Phi(s_{t+1})\right) + \cdots | s_t, a_t\right]$$
$$= \mathbb{E}_{\pi}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots | s_t, a_t] - \Phi(s_t) = Q^{\pi}_{M}(s_t, a_t) - \Phi(s_t)$$
$$V^{\pi}_{M'}(s_t) = \mathbb{E}_{a_t \sim \pi}\left[Q^{\pi}_{M'}(s_t, a_t)\right] = \mathbb{E}_{a_t \sim \pi}[Q^{\pi}_{M}(s_t, a_t) - \Phi(s_t)] = V^{\pi}_{M}(s_t) - \Phi(s_t)$$

  - $Q^{*}_{M'}(s,a) = Q^{*}_{M}(s,a) - \Phi(s)$, $V^{*}_{M'}(s) = V^{*}_{M}(s) - \Phi(s)$: optimal value function
- optimal policy invariant
  - $\pi^{*}_{M'} = \pi^{*}_{M}$: optimal policy to M' is also optimal to M
    - $\pi^{*}_{M'}(s) = \arg\max_a Q^{*}_{M'}(s,a) = \arg\max_a \{Q^{*}_{M}(s,a) - \Phi(s)\} = \arg\max_a Q^{*}_{M}(s,a) = \pi^{*}_{M}(s)$
- interpretation of potential based reward shaping
  - $\Phi(s) \approx V^{*}_{M}(s)$ ➔$V^{*}_{M'}(s) \approx 0$, $Q^{*}_{M'}(s,a) \approx Q^{*}_{M}(s,a) - V^{*}_{M}(s)$ ➔ but, $V^{*}_{M}(s)$ is unknown
    - Use some approximate value function as $\Phi(s)$ (from task specific knowledge)
    - Some heuristic policy π_h → evaluate $V^{\pi_h}_{M}$ → use $\Phi(s) = V^{\pi_h}_{M}(s)$
  - if $\Phi(s) \approx V^{*}_{M}(s)$, delayed reward → immediate reward
    - $R'(s,a,s') = R(s,a,s') + \gamma\Phi(s') - \Phi(s) \approx R(s,a,s') + \gamma V^{*}_{M}(s') - V^{*}_{M}(s)$

example : grid world

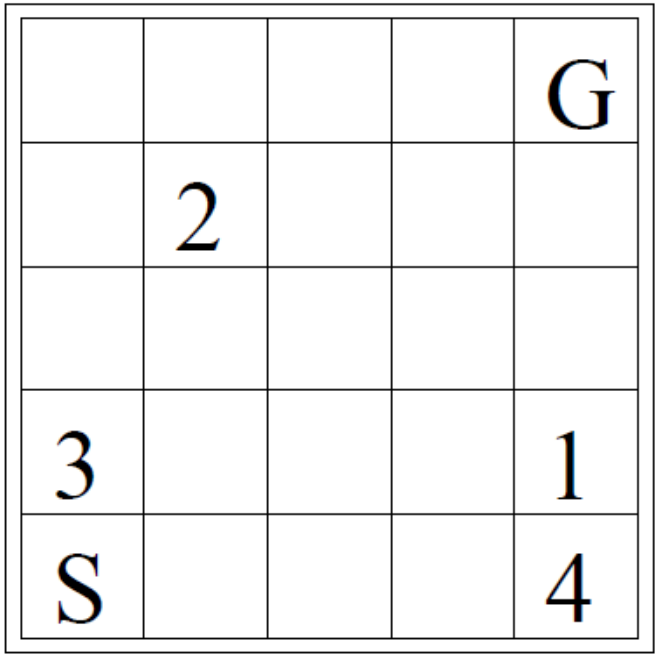
- $g \times g$ grid world $(g = 10, 50)$: start and goal at opposite corners
  - State transition after action (up, down, left, right)
    - move to intended direction w.p. 0.8, random direction w.p. 0.2
  - original reward = -1 per step, no discount ($\gamma = 1$)
- shaping reward
  - $\Phi_0(s) = -\text{Manhattan}(s, \text{Goal})/0.8$
  - No shaping (dot), $\Phi(s) = 0.5\Phi_0(s)$ (dot-dash), $\Phi(s) = \Phi_0(s)$ (solid)

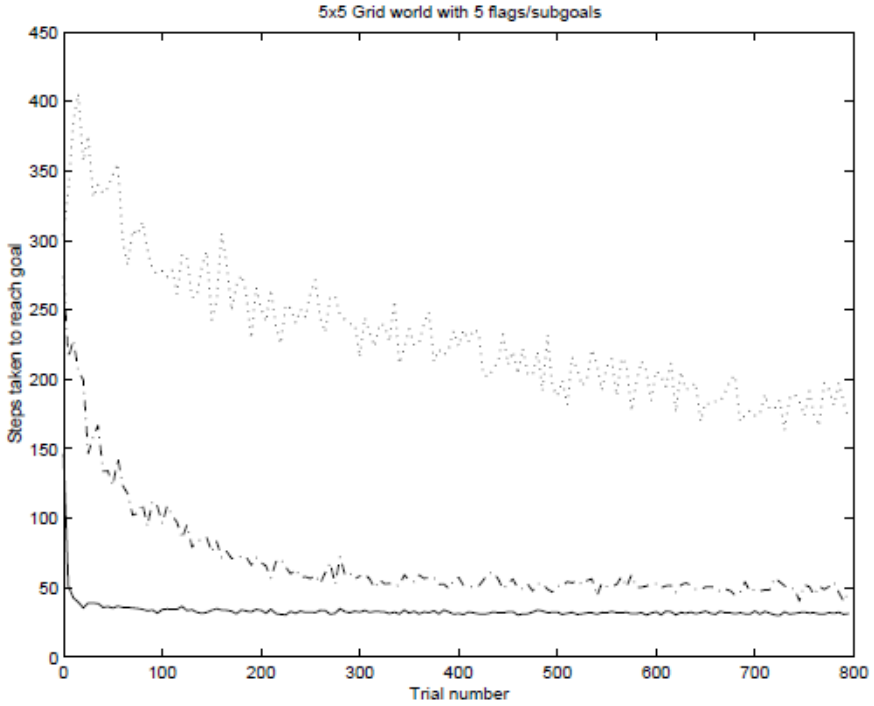




example : grid world with flags

- 5×5 grid world with 5 (numbered) flags : $\mathrm{S} \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow \mathrm{G}$
- state $s = (r, c, n)$, (r, c)=current location, n=flags collected so far
- shaping reward
  - $d(r, c) = \mathrm{Manhattan}((r, c), \mathrm{Goal})/0.8$: estimated distance to Goal
  - $r(n) = \frac{5-n-0.5}{5}$: portion of remaining flags (0.5 : s is halfway to next flag)
  - $\Phi_0(s) = -r(n)d(r, c)$
  - $\Phi_1(s)$: more fine-tuned estimate of remaining steps



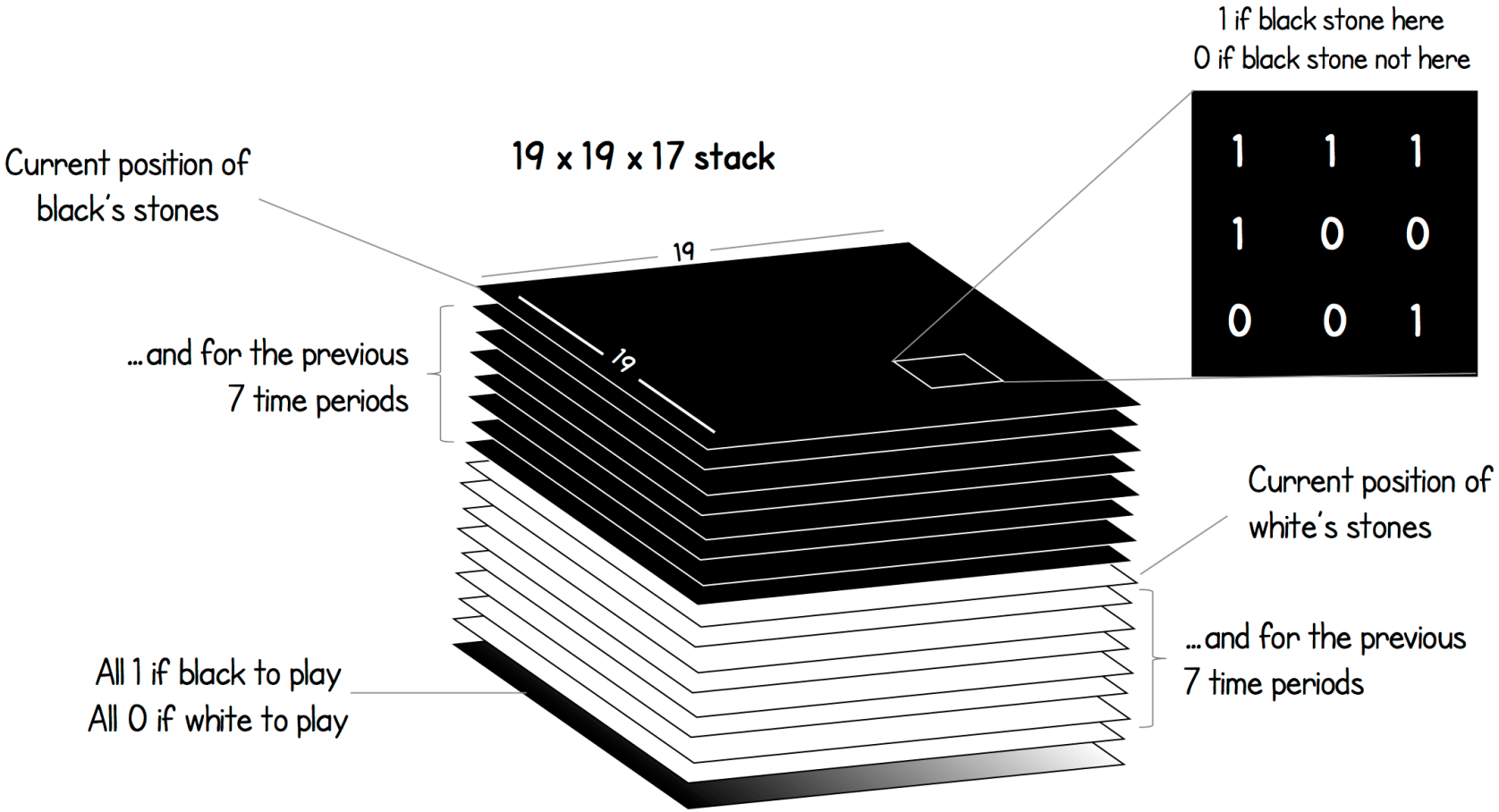


Summary of Reward Shaping

- Reward shaping : use $\Phi(s)$ to shape (change) reward
  - $R'(s,a,s') = R(s,a,s') + \gamma\Phi(s') - \Phi(s)$
    - state potential difference → immediate reward
    - i.e. delayed reward → immediate reward
  - hoping to learn $Q^*_{M'}(s,a)$ easily
    - without changing optimal policy
    - $Q^*_{M'}(s,a) = Q^*_M(s,a) - \Phi(s)$: advantage w.r.t. $\Phi(s)$
  - shaping reward helps if $\Phi(s)$ resembles $V^*_M(s)$
    - $\Phi(s)$ approximates state value
- TAQ. So, if we have a good potential function $\Phi(s)$,
  - instead of using $\Phi(s)$ for changing reward,
  - can we use it for initializing $V(s)$ or $Q(s,a)$?

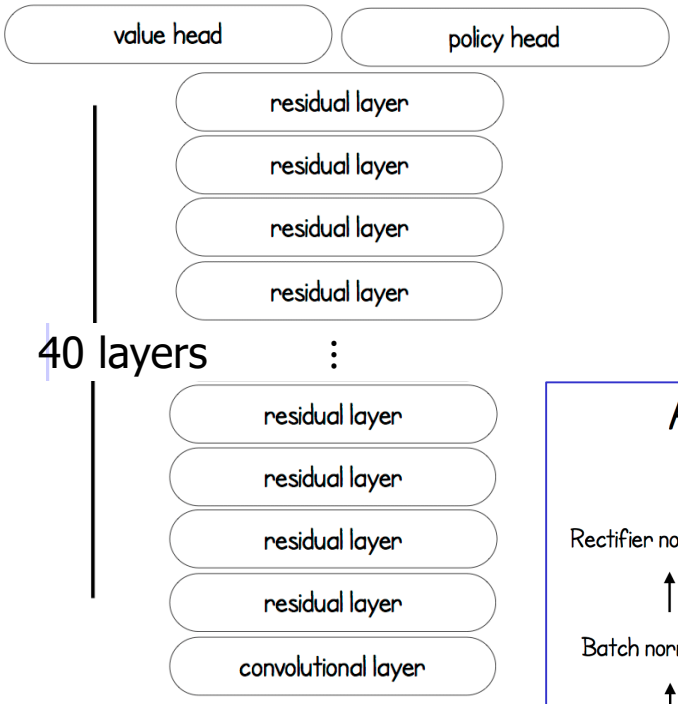
A1. AlphaGo Zero - Input

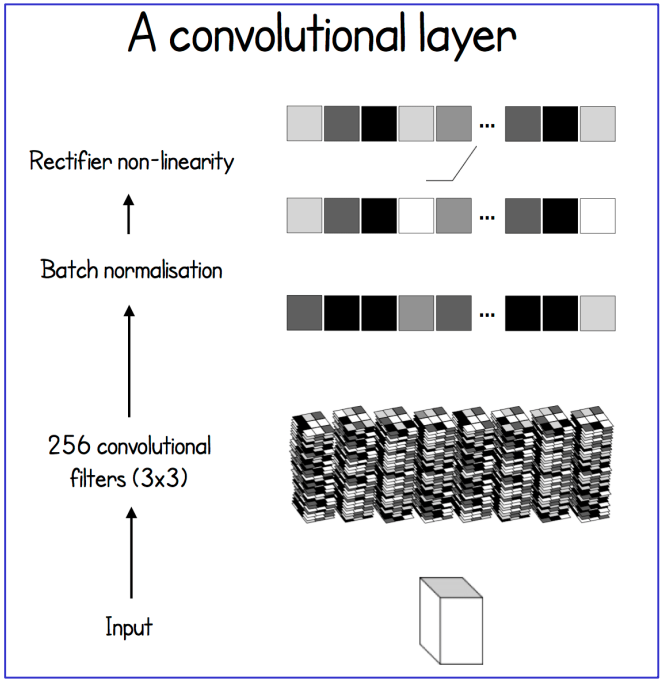
Input : 19x19x17 = (Current + Past 7)*2 players+1

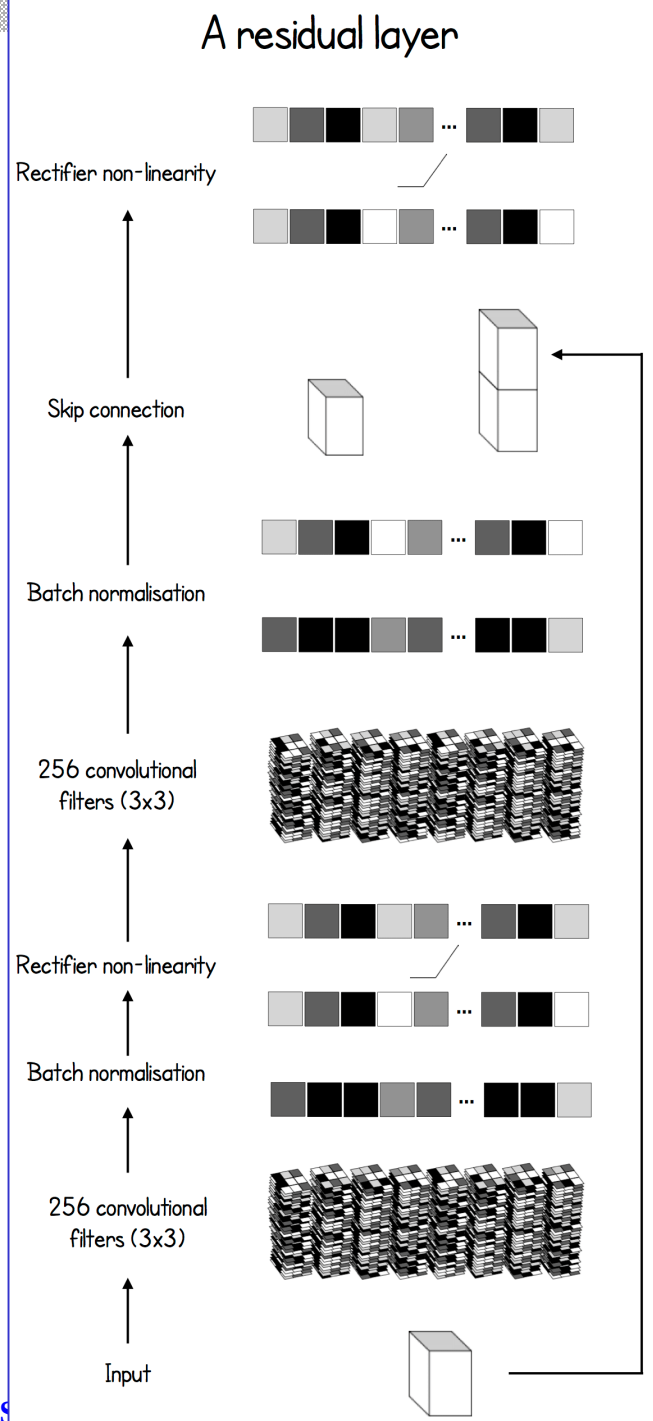


Source : "AlphaGo Zero Cheat Sheet",
https://medium.com/applied-data-science/alphago-zero-explained-in-one-diagram-365f5abf67e0

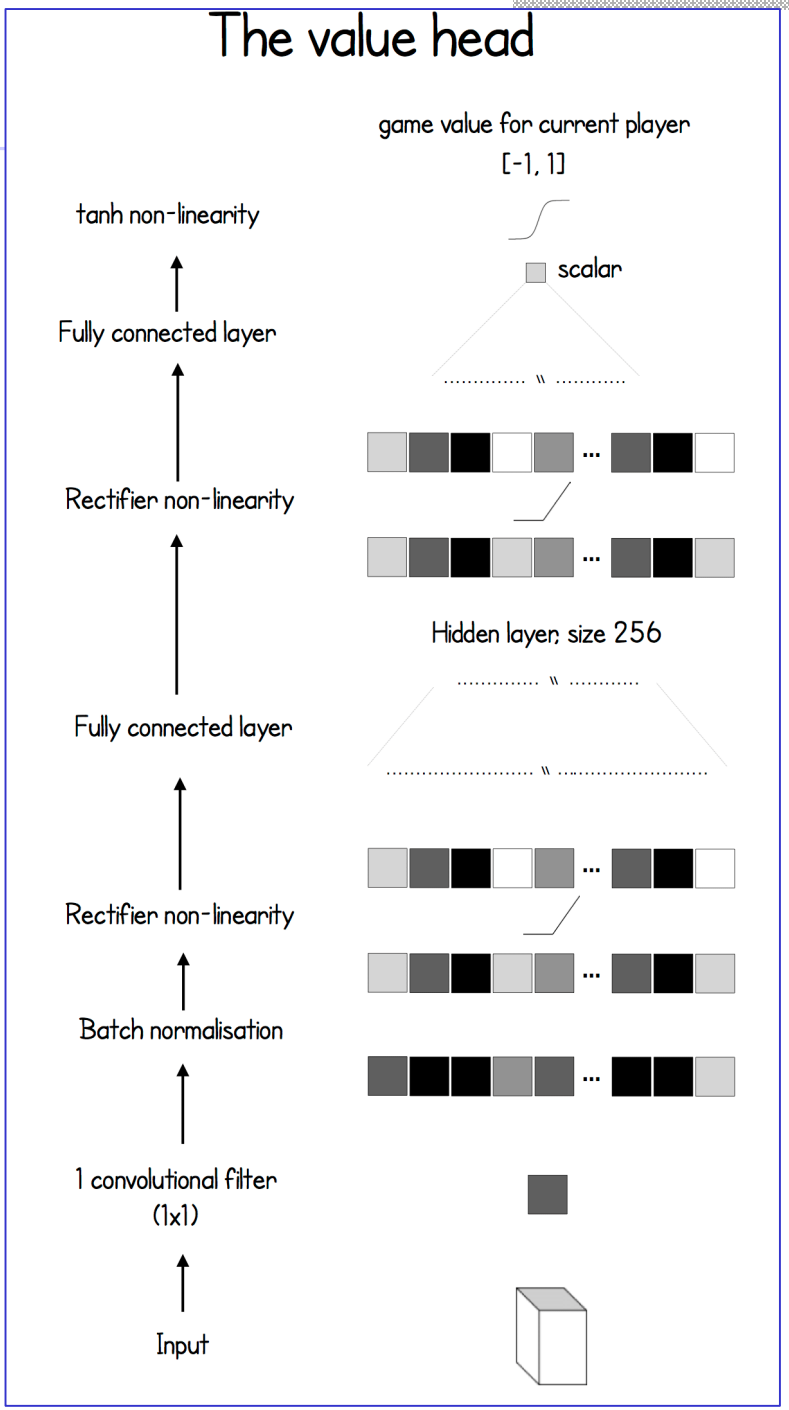
A1. AlphaGo Zero - net structure

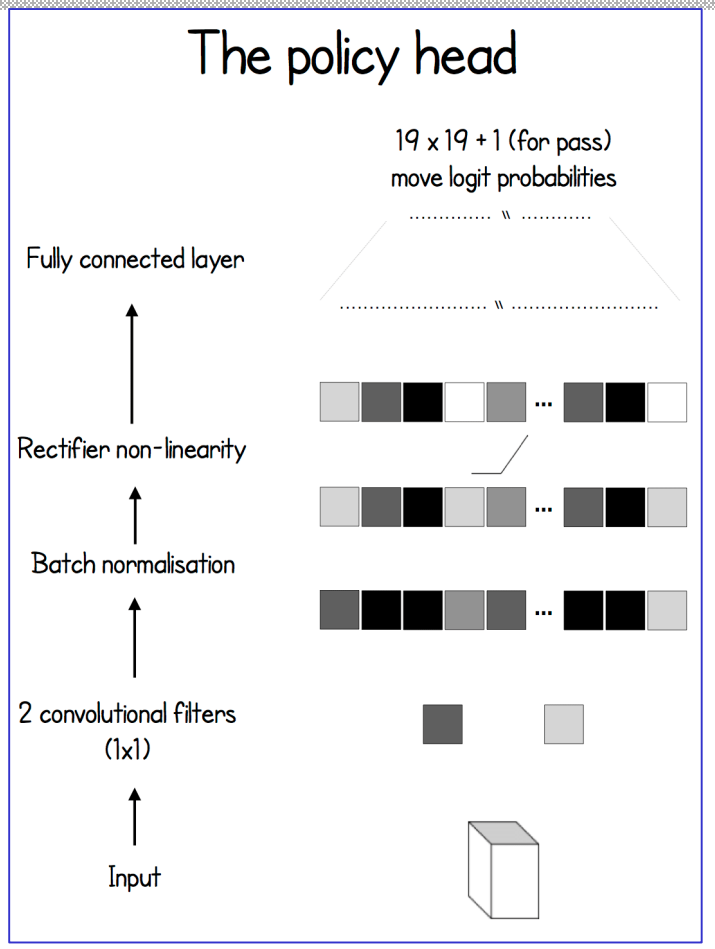






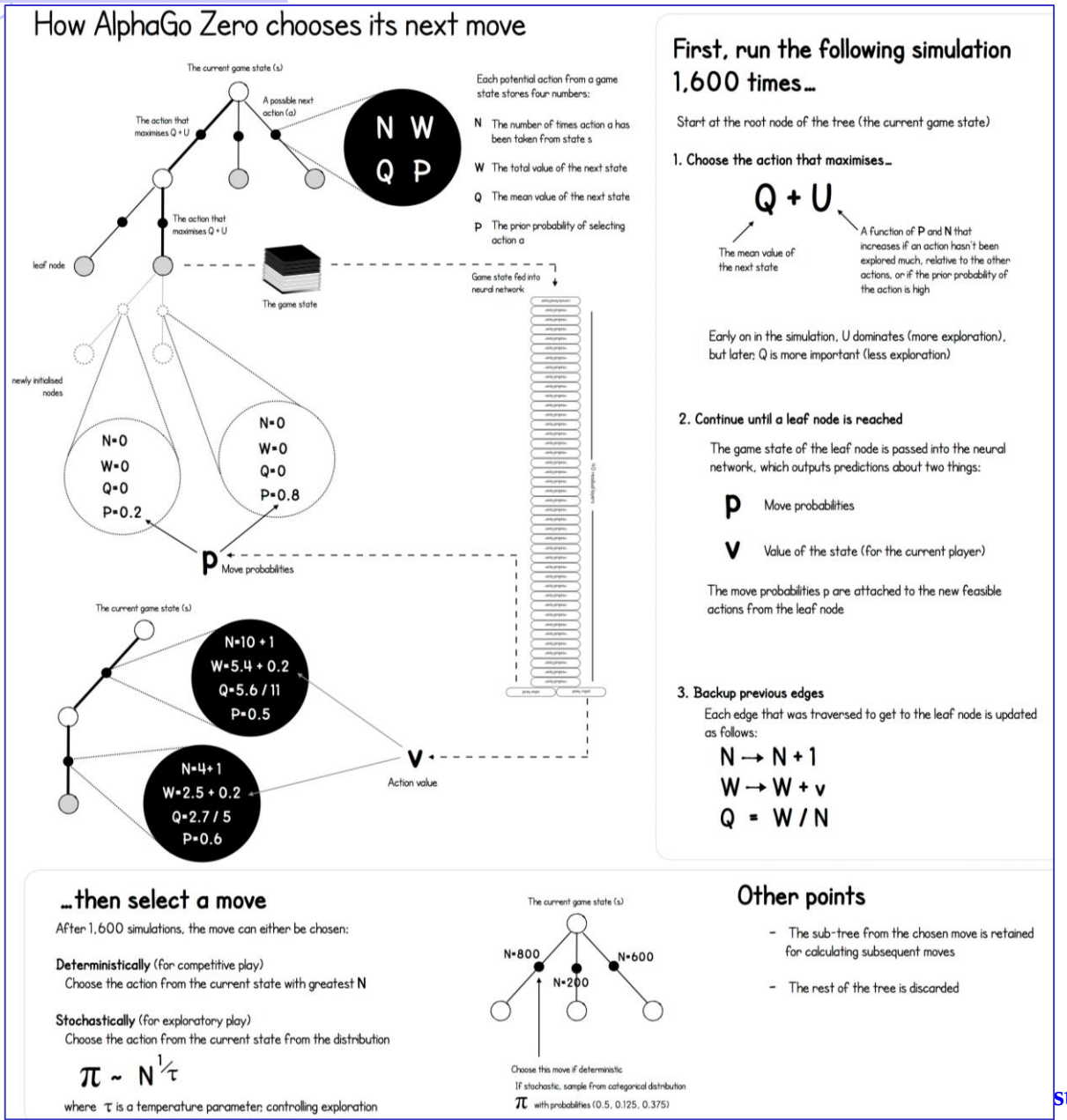
Source : "AlphaGo Zero Cheat Sheet"



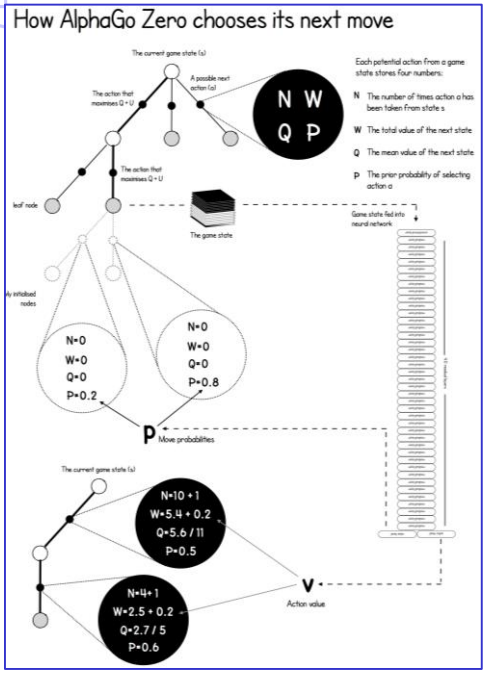


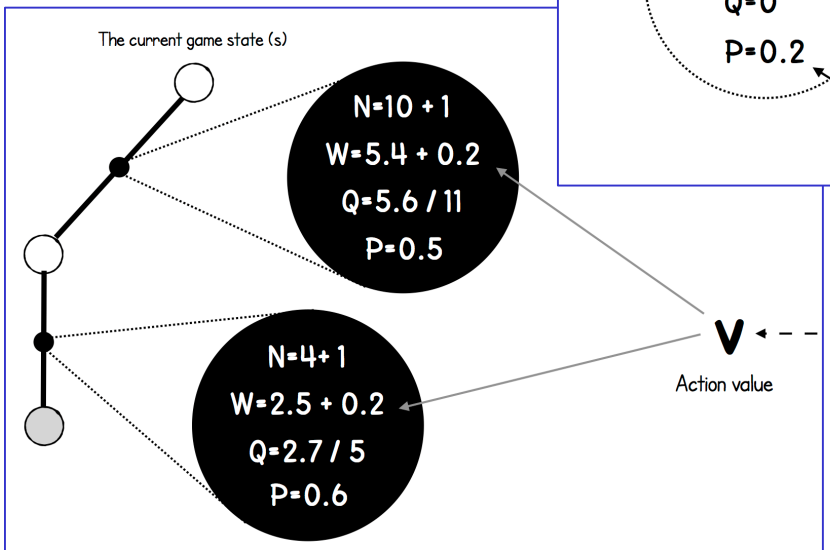
Source : "AlphaGo Zero Cheat Sheet"

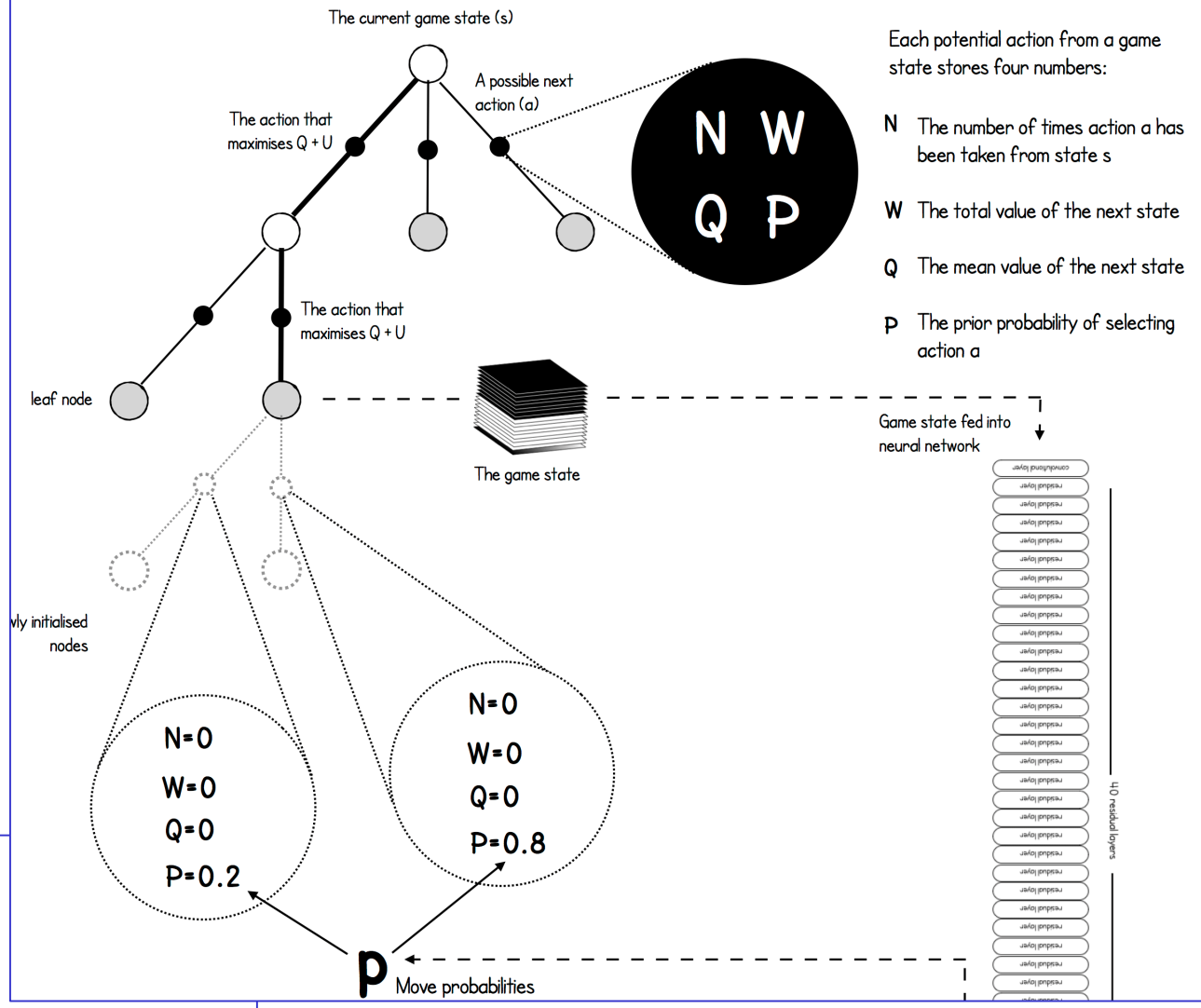
A1. AlphaGo Zero - MCTS



MCTS







Source : "AlphaGo Zero Cheat Sheet"

First, run the following simulation 1,600 times…

Start at the root node of the tree (the current game state)

1. Choose the action that maximises…

$$Q + U$$

- Q: The mean value of the next state
- U: A function of **P** and **N** that increases if an action hasn't been explored much, relative to the other actions, or if the prior probability of the action is high

Early on in the simulation, U dominates (more exploration), but later, Q is more important (less exploration)

2. Continue until a leaf node is reached

The game state of the leaf node is passed into the neural network, which outputs predictions about two things:

- $\mathbf{p}$ Move probabilities
- $\mathbf{v}$ Value of the state (for the current player)

The move probabilities p are attached to the new feasible actions from the leaf node

3. Backup previous edges

Each edge that was traversed to get to the leaf node is updated as follows:

$$N \rightarrow N + 1$$
$$W \rightarrow W + v$$
$$Q = W / N$$

…then select a move

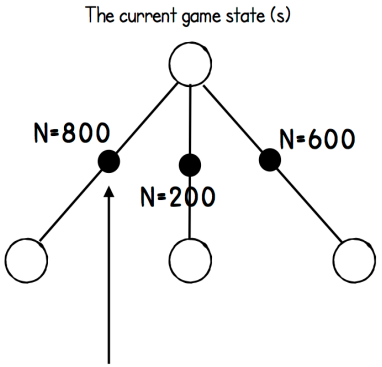
After 1,600 simulations, the move can either be chosen:

Deterministically (for competitive play)
Choose the action from the current state with greatest **N**

Stochastically (for exploratory play)
Choose the action from the current state from the distribution

$$\pi \sim N^{1/\tau}$$

where τ is a temperature parameter, controlling exploration



Other points

- The sub-tree from the chosen move is retained for calculating subsequent moves
- The rest of the tree is discarded